

Práctica 1 – SDR y GNURadio

Landazabal Reyes Andres Sebastian, andres2200540@correo.uis.edu.co

Saavedra Diaz Nicolás Felipe, nicolas2202788@correo.uis.edu.co

https://github.com/andreslandazabal/Comm2_A1_G8.git

Abstract

This work presents the implementation of custom Python blocks within a Software Defined Radio (SDR) environment using GNU Radio. The primary objective was to develop and evaluate Digital Signal Processing (DSP) algorithms in real time, specifically focusing on an accumulator block and a differentiator block.

The methodology employed consisted of creating signal flows for data generation, alongside the use of Python Blocks for the logical implementation of the algorithms. Tests were conducted using both pure sinusoidal signals and signals affected by noise, allowing for a dynamic behavior analysis of the implemented systems.

The results demonstrate that the differentiator emphasizes high-frequency components and amplifies noise, while the accumulator acts as a discrete integrator, generating a progressive growth or ramp in the signal. Based on these findings, it can be inferred that programming custom blocks in SDR constitutes an effective tool for experimentation and advanced analysis of communication systems.

1. Introducción

La radio definida por software (SDR) es un sistema de radiocomunicación en el que muchos de los componentes de hardware tradicionales, como filtros, mezcladores, moduladores y demoduladores, se sustituyen por software que se ejecuta en un ordenador o en un procesador integrado. Esto nos aporta una mejor adaptabilidad en comparación con la radio tradicional, ya que permite un desarrollo más rápido y una comunicación más flexible entre distintos servicios y estándares.

Algunos componentes clave en la SDR son la antena y el hardware encargado de transmitir y recibir las señales, así como los convertidores analógico-digital (ADC) y digital-analógico (DAC), ya que estos permiten convertir señales entre el dominio analógico y el digital, haciendo posible su procesamiento por software.

Para desarrollar este tipo de sistemas se utiliza GNU Radio, que es un conjunto de herramientas de desarrollo que proporciona bloques de procesamiento de señales para la implementación de radios definidas por software. Esto facilita tanto la simulación como la implementación práctica, ofreciendo un soporte más flexible que otros sistemas. Además, incluye bloques programables, como el bloque de Python, los cuales permiten implementar funciones personalizadas que no se encuentran de manera nativa dentro del entorno.

2. Objetivos

Nuestro objetivo principal consistió en el uso de la herramienta GNURadio y textos guías con el fin de implementar bloques al programa para el análisis de señales, siendo estos un acumulador y un diferenciador, con el fin de revisar su comportamiento sin y con un ruido añadido.

Como objetivo secundario nos planteamos el uso de GitHub como una herramienta que nos permitiera trabajar de manera conjunta pero individual con su sistema de “Branchs”.

3. Metodología

Para el correcto desarrollo de la práctica, se trabajó utilizando un sistema de control de versiones mediante Git, alojando el repositorio en la plataforma GitHub. Esto permitió organizar el trabajo colaborativo y mantener un seguimiento estructurado de los cambios realizados durante el desarrollo del laboratorio.

Inicialmente, se creó un repositorio principal dentro del cual se generó la rama denominada Practica_I, destinada a evidenciar los avances de la práctica. Posteriormente, cada integrante del grupo creó una rama individual a partir de esta, con el fin de desarrollar sus propios algoritmos sin interferir con el trabajo de los demás. Para la gestión del repositorio, se utilizaron comandos como git clone, git checkout, git add, git commit y git push, permitiendo sincronizar los cambios entre el entorno local y la nube.

Desarrollo del acumulador

Para el desarrollo del sistema de procesamiento se utilizó el software GNU Radio, donde se implementó un diagrama de bloques para la generación, procesamiento y visualización de señales.

Implementación del bloque Acumulador

En la aplicación de GNU Radio se creó un diagrama de bloques compuesto por:

- Un bloque Signal Source: genera una señal senoidal de 1 kHz, la cual tiene una frecuencia de muestreo de 32 kHz.

- Un Python Block: se personalizó para que actuara como acumulador.
- Dos bloques QT GUI TIME Sink: utilizados para visualizar tanto la señal original como la señal procesada en el dominio del tiempo.

Para el acumulador, basándose en el libro guía y con ayuda para la corrección de errores, se implementó el código que se puede observar en la figura 1:

```
1 import numpy as np
2 from gnuradio import gr
3
4 class blk(gr.sync_block):
5
6     def __init__(self):
7         gr.sync_block.__init__(
8             self,
9             name="e_Acum",
10            in_sig=[np.float32],
11            out_sig=[np.float32]
12        )
13
14        # Estado interno (memoria)
15        self.acum_anterior = np.float32(0)
16
17    def work(self, input_items, output_items):
18
19        x = input_items[0]
20        y0 = output_items[0]
21
22        N = len(x)
23        if N == 0:
24            return 0
25
26        # Acumulado continuo
27        cumsum_local = np.cumsum(x)
28        y0[:] = cumsum_local + self.acum_anterior
29
30        # Actualizar estado para siguiente buffer
31        self.acum_anterior = y0[N - 1]
32
33        return N
```

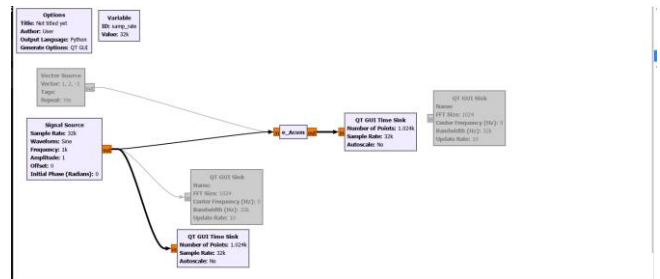
Figura 1. Código acumulador.

El bloque acumulador fue implementado para señales de tipo `np.float32`, tanto para la entrada como para la salida. Se definió, además, una variable interna llamada `acum_anterior`, la cual actúa como una memoria para el sistema. Dentro del método `work()`, se reciben los vectores correspondientes a la entrada y la salida, y se realiza una suma acumulativa utilizando `np.cumsum()`, sumando el valor acumulado de la iteración anterior. Para finalizar, se devuelve el estado del último valor calculado, permitiendo así que el proceso continúe de manera correcta.

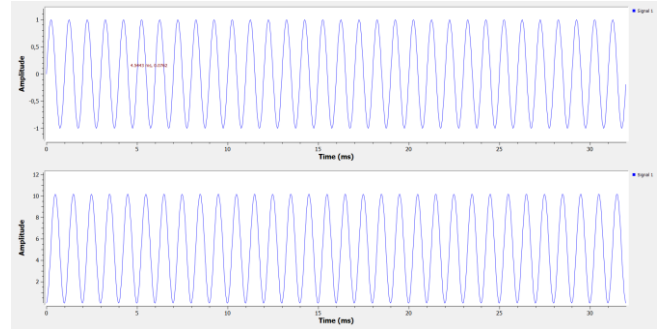
Este procedimiento asegura un acumulado continuo en tiempo real, respetando la naturaleza de procesamiento por bloques del entorno GNU Radio.

4. Análisis de resultados

Una vez terminado el código procedimos a montar un sistema simple en GNURadio el cual consiste en una fuente de señal que se conecta a nuestro acumulador y este llega a un visualizador quedando así de la siguiente forma.

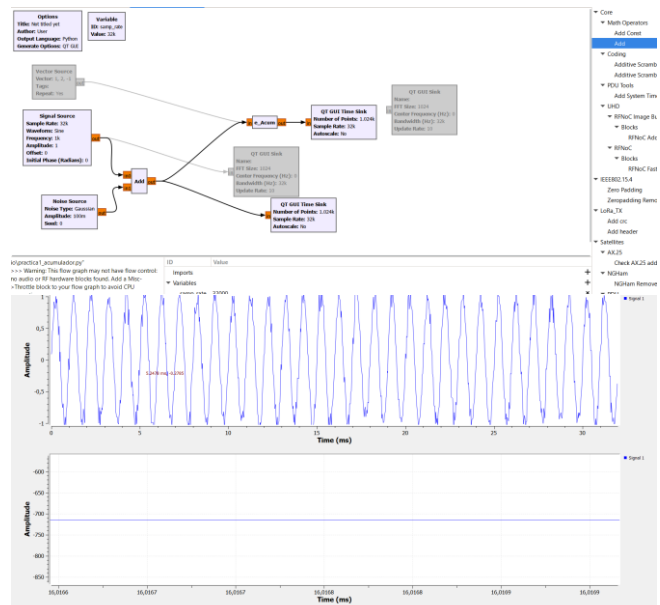


Una vez implementado se obtuvo la siguiente grafica del bloque QT GUI Time Sink.



Acumulador sin ruido

Como se puede observar este último funciona sin problemas y nos permite ver la señal a lo largo del tiempo, siguiendo esto decidimos de implementar una entrada de ruido para analizar su comportamiento, la implementación y el resultado se muestran a continuación



Acumulador con ruido

Como logramos ver la señal con ruido presenta variaciones rápidas en la amplitud debido a que el ruido blanco gaussiano introduce componentes de alta frecuencia, generando cambios abruptos entre muestras consecutivas. Estas variaciones son inherentes a la naturaleza estadística del ruido y se hacen evidentes al superponerse sobre la señal senoidal original.

5. Conclusiones

La importancia de GNURadio como software de análisis en comunicaciones no es solo debido a las funciones que tiene por defecto el programa, sino a lo adaptable que es al permitir el codificado de nuevos bloques como se evidencio en la práctica al crear los bloques solicitados.

Algo de suma importancia a la hora de usar el programa, es tener en cuenta el entorno ideal en el que nos encontramos, aunque podamos de obtener resultados a los experimentos en el software debemos siempre recordar que estos no se vieron afectados por el ruido, por lo cual es indispensable el realizar siempre los experimentos con una fuente de ruido con el fin de no solo llegar a resultados más reales, sino paa ver que tan bien nuestro sistema codificado reaccionara ante este último.

Referencias

- [1] Assured-systems.com. [En línea]. Disponible en: <https://www.assured-systems.com/es/faq/what-is-software-defined-radio-sdr-and-how-does-it-work/>. [Consultado: 25-feb-2026]